

## API SPECIFICATION DOCUMENT

### PART-1: API FOUNDATION

# School ERP

Spring Boot 3 + Java 21 | Spring Security + JWT | React + TypeScript

Prepared by: API Architecture Team

Document Version: 1.0

Date: 29 July 2026

*Classification: Confidential — Engineering Team Only*

## 1. Introduction

---

This document is the API Specification Document (ASD) v1.0, Part-1: API Foundation, for the School ERP System. It is the formal, standalone API architecture reference consolidating the logical API design already established in Appendix-K (API Specification, Logical) with the concrete Spring Boot 3/Java 21 realization already fixed in Master Development Blueprint (MDB) Part-2, now organized specifically around API architecture, URI, HTTP method, versioning, content-type, and request/response conventions.

This document does not regenerate RGD v2.0, any prior Appendix, any UI/UX Screen Specification, any MDB Part, or any Database Design Document (DDD) Part. It contains no source code, no controller code, no API endpoint list, no JSON examples, no OpenAPI/Swagger definition, and no sequence diagrams — it fixes conventions and standards, not a concrete, enumerable interface contract.

Authentication, Authorization, and JWT lifecycle standards remain fixed in MDB Part-2 Sections 11-13 and are not restated here except where directly relevant to a request/response convention (Section 9-10) — this ASD's scope is API architecture and design convention, not security, which has its own authoritative document.

## 2. API Objectives

---

- Provide a single, consistent interface convention across all 11 modules' API Groups already catalogued in Appendix-K Section 18, so that no module's API deviates stylistically from another's.
- Realize every conceptual API Design Principle already fixed in Appendix-K Section 3 as a concrete, Spring Boot-specific convention, without altering any principle's substance.
- Ensure every API operation remains traceable to the Functional Requirement, Business Rule, and Screen ID it serves, extending the traceability discipline already fixed throughout this documentation set into the API layer itself.
- Support the Performance and Scalability targets already fixed in Appendix-F through API-level conventions (pagination, versioning, content negotiation) that do not themselves become a bottleneck.
- Provide a stable foundation that a frontend team (React + TypeScript) and backend team (Spring Boot 3) can each build against independently, using this document as the shared contract convention rather than ad hoc, per-feature negotiation.

### 3. API Architecture

---

Confirms, without altering, the API's position within the layered architecture already fixed in MDB Part-1 Section 4 and Appendix-I Section 7.

- Every API operation is served by the Spring Boot 3 Application Layer's Controller components (MDB Part-1 Section 6), which delegate immediately to Service components and perform no business logic themselves, per MDB Part-3 Section 24.
- The API is the sole interface between the React + TypeScript frontend (MDB Part-1 Section 7) and the backend — the frontend never queries PostgreSQL or Redis directly, and no backend module bypasses the API layer to serve a screen.
- Every API Group already catalogued in Appendix-K Section 18 maps to exactly one backend module package (MDB Part-1 Section 6) — an API Group is never implemented by logic spanning multiple module packages directly, consistent with the module-boundary discipline already fixed in MDB Part-1 Section 8.
- The API is stateless (Appendix-K Section 3, MDB Part-2 Section 11) — no server-side session state beyond the JWT-carried identity is required to process any request, enabling the horizontal scaling already fixed in MDB Part-1 Section 4.

## 4. REST Design Principles

Fixes the concrete REST convention realizing the conceptual principles already established in Appendix-K Section 3.

- Every API Group is resource-oriented, centered on the business entity or capability it exposes (Appendix-G/H entities, Appendix-K Section 18 API Groups) — an API Group is never organized around a screen or a single workflow step.
- Standard HTTP semantics (Section 6) are used exactly as intended — an operation's safety and idempotency characteristics match its HTTP method, rather than, for example, a state-changing action being exposed through a GET request.
- HATEOAS (hypermedia-driven response navigation) is not adopted — clients navigate the API using the URI conventions fixed in Section 5 and the module/screen mapping already established in the UI/UX Screen Specifications, not response-embedded links.
- Idempotency-sensitive operations (Payment recording, Admission confirmation — Appendix-K Section 14, MDB Part-2 Section 14) use an idempotency-key convention regardless of which HTTP method they use, since idempotency is a business property, not solely a property of the HTTP method chosen.

## 5. URI Naming Standards

Fixes the definitive URI convention, consolidating MDB Part-1 Section 9 and Appendix-K Section 3 — no specific endpoint is enumerated here, per this document's scope.

| Element               | Convention                                                                                                                                                                                                                              |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Resource Path Segment | kebab-case, plural noun, matching the owning entity/API Group (e.g., a path segment for Fee Structures, not FeeStructure or feeStructure)                                                                                               |
| Nested/Sub-Resource   | Path nesting reflects a genuine ownership relationship already fixed in Appendix-H (e.g., a sub-resource of a Student for that student's documents), not an arbitrary grouping                                                          |
| Path Parameter        | Identifies a specific resource instance by its surrogate ID (DDD Part-1 Section 9), never by a natural/business key                                                                                                                     |
| Query Parameter       | Used for filtering (Appendix-K Section 10), sorting (Section 11), pagination (Section 9), and search (Section 12) — never for identifying a specific resource                                                                           |
| Action-Style Endpoint | Reserved for operations that are not pure CRUD (e.g., a Payroll Processing run, a Report Card publish action) — expressed as a sub-resource or clearly-scoped action path, not a verb inserted into an otherwise resource-oriented path |

## 6. HTTP Method Standards

Fixes the mapping between HTTP methods and the Create/Read/Update/Delete/Search operations already defined per API Group in Appendix-K Section 18.

| HTTP Method | Operation                                                                     | Safety / Idempotency                                                                                                                  |
|-------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| GET         | Read a single resource or Search/List a collection (Appendix-K Sections 9-12) | Safe and idempotent — never used for a state-changing operation                                                                       |
| POST        | Create a new resource, or invoke an action-style endpoint (Section 5)         | Not idempotent by default; idempotency-key convention applied where Section 4 requires it                                             |
| PUT         | Full replacement update of a resource                                         | Idempotent by definition — repeating the same PUT produces the same end state                                                         |
| PATCH       | Partial update of a resource                                                  | Used where only a subset of fields changes, consistent with the UI/UX Screen Specifications' per-field Editable/Read-Only conventions |
| DELETE      | Soft-delete a resource (DDD Part-1 Section 7)                                 | Idempotent — repeating a DELETE against an already-soft-deleted resource does not error                                               |

## 7. API Versioning Strategy

---

Fixes the concrete versioning mechanism realizing the conceptual approach already established in Appendix-K Section 13.

- API versioning uses a leading URI path segment (e.g., a version identifier preceding the resource path) — this is the sole versioning mechanism; header-based or content-negotiation-based versioning is not used, avoiding ambiguity between the two.
- A version increment follows the Backward Compatibility rules already fixed in Appendix-K Section 24 exactly — an additive, non-breaking change (a new optional field, a new endpoint) does not require a version increment; a breaking change (a removed field, a changed field meaning, a newly-mandatory input) does.
- Multiple API versions may run concurrently during a deprecation window, consistent with the API Lifecycle already fixed in Appendix-K Section 23 — a version is not retired the moment its successor is released.
- Version-specific behavior is isolated at the Controller layer (MDB Part-1 Section 6) — Service-layer Business Rule enforcement (MDB Part-3 Section 24) does not fork by API version, since Business Rules are a single source of truth regardless of which API version a client is using.

## 8. Content Type Standards

---

- JSON is the sole content type for all request and response bodies except File Upload operations (Appendix-K Section 16, MDB Part-2 Section 18), which use the standard multipart content type — no other content type (XML, form-encoded) is supported.
- Every JSON request/response uses a consistent character encoding, avoiding encoding-related data corruption for the multilingual content already anticipated in Appendix-F's Localization NFR series (Hindi and a regional language, per RGD v2.0).
- Generated-document downloads (report cards, receipts, payslips — Appendix-I Section 12.2) are served with the appropriate document content type (e.g., PDF), distinct from the JSON convention governing the API's data operations.
- Content-type negotiation is not used to select between multiple possible response formats for the same endpoint — each endpoint has exactly one response content type, keeping client implementation unambiguous.



## 9. Request Standards

---

Fixes the definitive request-element convention, consolidating Appendix-K Section 6 and MDB Part-2 Section 15.1 without reproducing example payloads.

- Every authenticated request carries the JWT access token exactly as already fixed in MDB Part-2 Section 13, and a unique request identifier exactly as already fixed in MDB Part-2 Section 15.1 — these two elements are non-negotiable on every request regardless of API Group.
- Pagination, filtering, and sorting parameters (Appendix-K Sections 9-11) use the exact field names already defined per entity in Appendix-G's Attribute Catalogue — a query parameter name never diverges from its corresponding database column's logical name.
- Request payload structure for Create/Update operations mirrors the Form Fields already defined per screen in the relevant UI/UX Screen Specification — mandatory/optional status, and validation rules, match exactly between the screen specification and the API request contract.
- A request exceeding the API's defined size or rate limits (MDB Part-2 Section 14, Appendix-F NFR-APIP-001) is rejected before reaching Service-layer logic, consistent with the Rate Limiting standard already fixed.

## 10. Response Standards

---

Fixes the definitive response-element convention, consolidating Appendix-K Section 7 and MDB Part-2 Section 15.2 without reproducing example payloads.

- Every response carries the status indicator already fixed in Appendix-K Section 8/MDB Part-2 Section 15.2, distinguishing success, validation failure, business-rule failure, authorization failure, and system error — a client can determine outcome category without parsing response body content heuristically.
- List/Search responses carry pagination metadata (current page, total count, page size) exactly as already fixed in Appendix-K Section 9, alongside the result set — never a bare array with pagination state omitted.
- Reports module responses additionally carry the Data Currency Indicator (last-refresh timestamp, provisional-data flag) already fixed in Appendix-K Section 7, directly operationalizing BR-RPT-004 — this is not optional for any Reports-module endpoint.
- Response bodies are always DTOs (MDB Part-1 Section 10.1, MDB Part-2 Section 15.2), never JPA entities (DDD Part-1) — this remains the single most important response-layer rule, since it is the primary safeguard against accidental exposure of internal persistence structure or an unintended Sensitive field.